Optimization
October 28th, 2026

Some of these slides are based on material from Professor Jonathan Kao, who also developed it based on other freely available sources including from Stanford and other resources, whom we acknowledge. Other slides on edge of stability and muon were done independently using recent results and papers.

Announcements

- HW #4 due Sunday November 1st 11.59pm uploaded to Gradescope. **Note that even 1 minute late is considered a day late! Your responsibility to confirm upload of HWs**
- HW#1, 2, 3 solutions posted.
- **Midterm 4th November 2026** (MSOL Midterm November 7th, Saturday); allowed one sheet (letter size), no devices or phones allowed in exam. We will have 2 rooms for the exams and will announce who will go where.

BH 8500

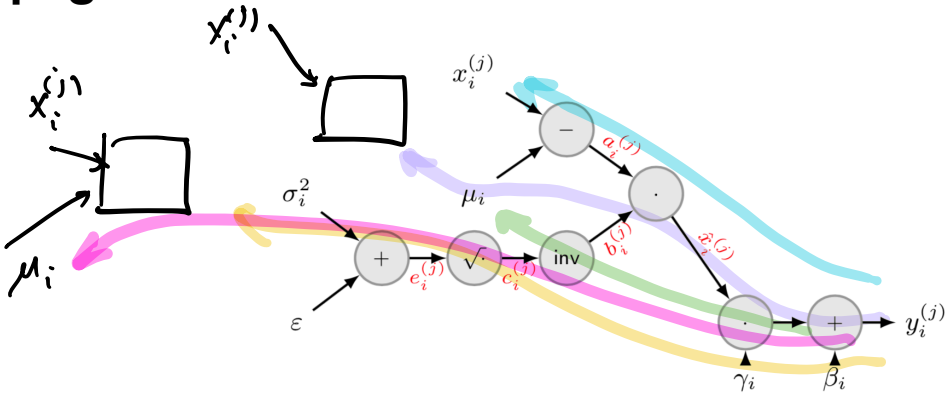
- Practice midterms will be posted soon, and solutions posted a week later.
- **Please read the rules handout carefully including for late HWs etc.**
- To submit your Jupyter Notebook, print the notebook to a pdf with your solutions and plots filled in. The graders will grade your work from these notebooks. In the future, you will also have to print out code from any .py files that were modified.
- Discussion sessions held on Fridays 11am, will be recorded and live zoom link also is available. **Midterm review on Friday, October 30th.**
- For midterm **CAE approved accommodation students have to contact CAE office to proctor exam** as we cannot do this ourselves. Please do this pro-actively.

Review of previous lecture

- **Computational graph and back propagation for batch normalization**

$$\mu_i = \frac{1}{m}\sum_{j=1}^{m} x_i^{(j)}$$

$$\sigma_i^2 = \frac{1}{m}\sum_{j=1}^{m} (x_i^{(j)} - \mu_i)^2$$



- **Regularization and avoiding overfitting**
  - Modifying loss function to account for inductive bias (e.g., $\ell_2, \ell_1, \ldots$ regularizations)
  - Dataset augmentations → encoded invariances
  - Ensemble methods: bagging, dropout
  - Multi-task learning
  - Transfer learning and parameter efficient fine-tuning
  - Early stopping…

explicit regularization

$$\tilde{J}(\underline{\theta}) = J(\underline{\theta}) + \lambda \Omega(\underline{\theta})$$

$\Omega(\underline{\theta})$

$\mathcal{D} \to \mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \ldots \mathcal{D}^{(K)}$

ℓ_2-regularization

$$\underline{\theta}_{t+1} = (1-\eta\lambda)\underline{\theta}_t - \eta \nabla J(\underline{\theta}_t)$$

$$\tilde{J}(\underline{\theta}) = J(\underline{\theta}) + \lambda \frac{1}{2}\|\underline{\theta}\|_2^2$$

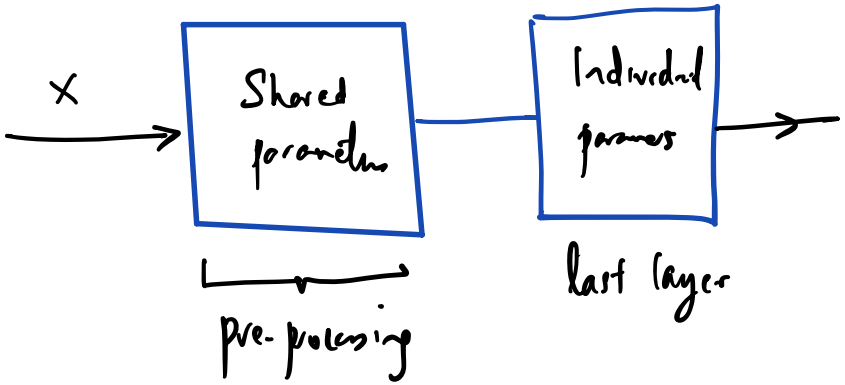
$$\nabla \tilde{J}(\underline{\theta}) = \nabla J(\underline{\theta}) + \lambda \cdot \underline{\theta}$$

$$\underline{\theta}_{t+1} = \underline{\theta}_t - \eta \nabla \tilde{J}(\underline{\theta}_t)$$
$$= \underline{\theta}_t - \eta [\nabla J(\underline{\theta}_t) + \lambda \underline{\theta}_t]$$

- **Momentum**

$$v \leftarrow \alpha v - \epsilon g \qquad \theta \leftarrow \theta + v$$

Review of previous lecture (Contd.)



- In transformers Low Rank Adaptors per layer (LoRA)
- Backbone + small # of parameters
- hypernetworks

parameter efficient fine-tuning (form of transfer learning)

Momentum

In momentum, we maintain the running mean of the gradients, which then updates the parameters.

Initialize $\mathbf{v} = 0$. Set $\alpha \in [0, 1]$. Typical values are $\alpha = 0.9$ or 0.99. Then, until stopping criterion is met:

- Compute gradient: $\mathbf{g}$
- Update:

maintain dependence on the history of gradients

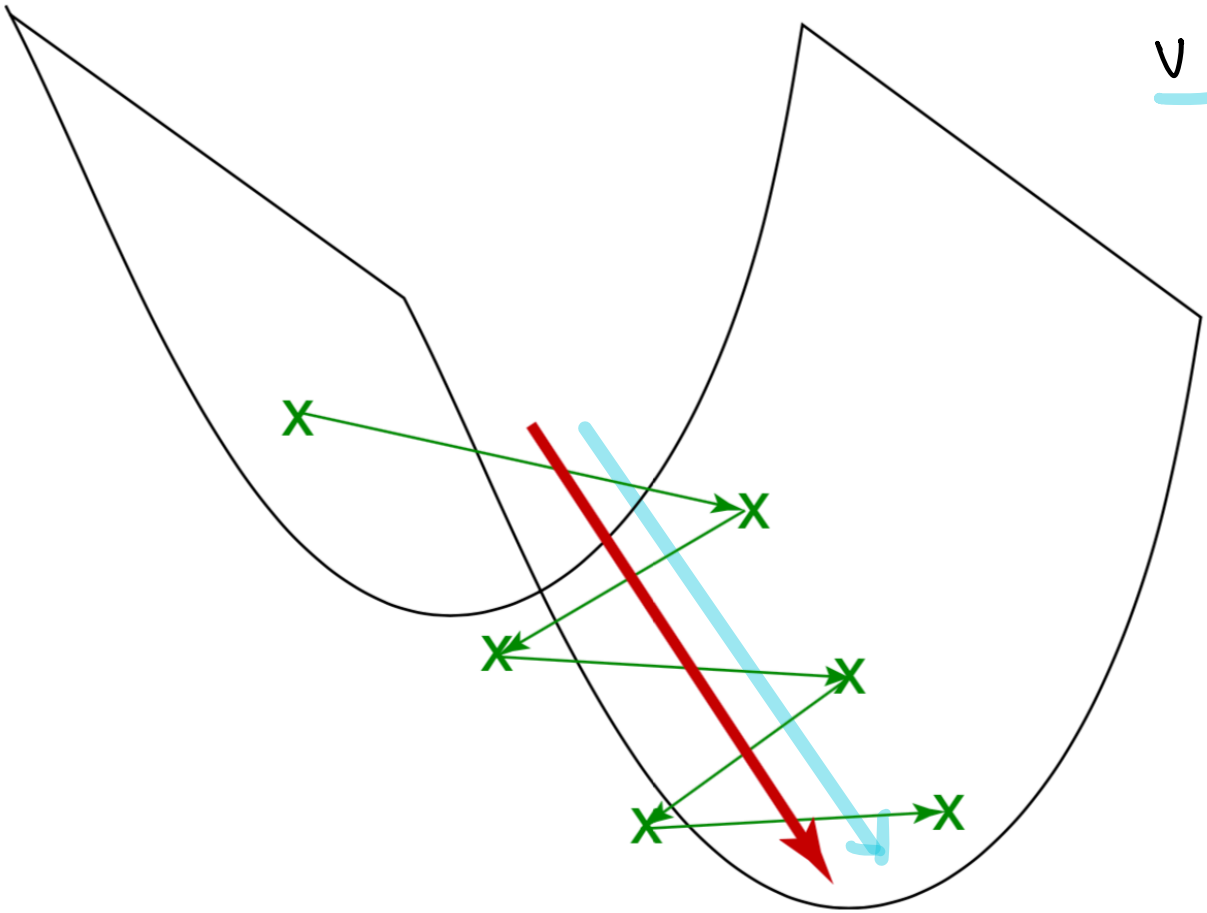
$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \varepsilon \mathbf{g}$$

- Gradient step:

$$\theta \leftarrow \theta + \mathbf{v}$$

Momentum (cont.)

An example of how momentum is useful is to consider a tilted surface with high curvature. Stochastic gradient descent may make steps that zigzag, although in general it proceeds in the right direction. Momentum will average away the zigzagging components.



"step size"

$$v_{t+1} = \alpha v_t - \epsilon g_t$$

$$v \leftarrow \alpha v - \epsilon g$$

$$\theta \leftarrow \theta + v$$

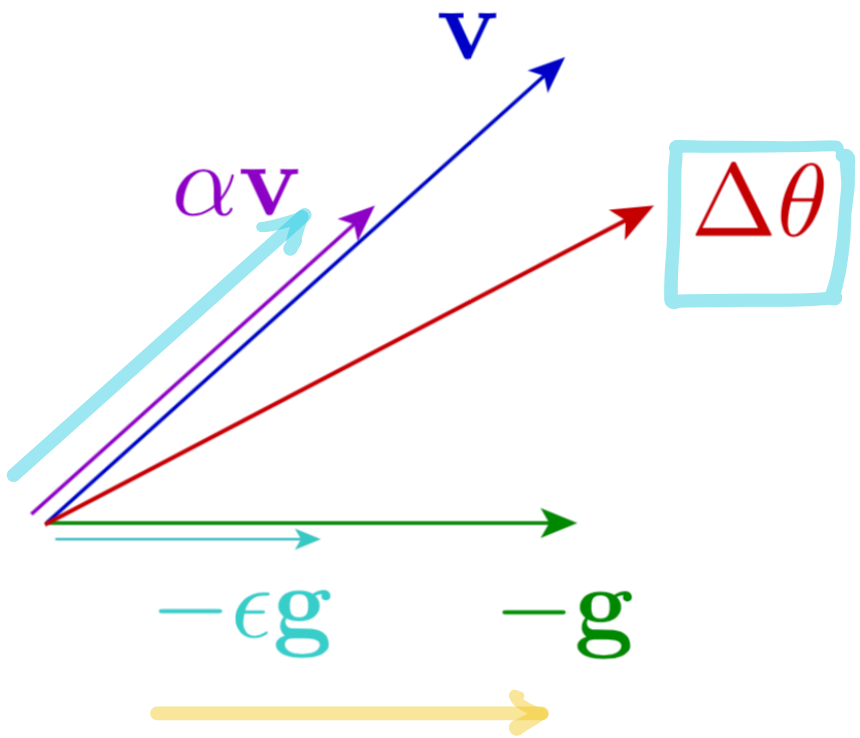
$$\theta_t + \alpha v - \epsilon g$$

$$\theta_{t+1} = \theta_t + \alpha v_t - \epsilon g_t$$

$$g_t = \nabla J(\theta_t)$$

Momentum (cont.)

This modification augments the gradient with the running average of previous gradients, which is analogous to a gradient "momentum." The following image is appropriate to have in mind:



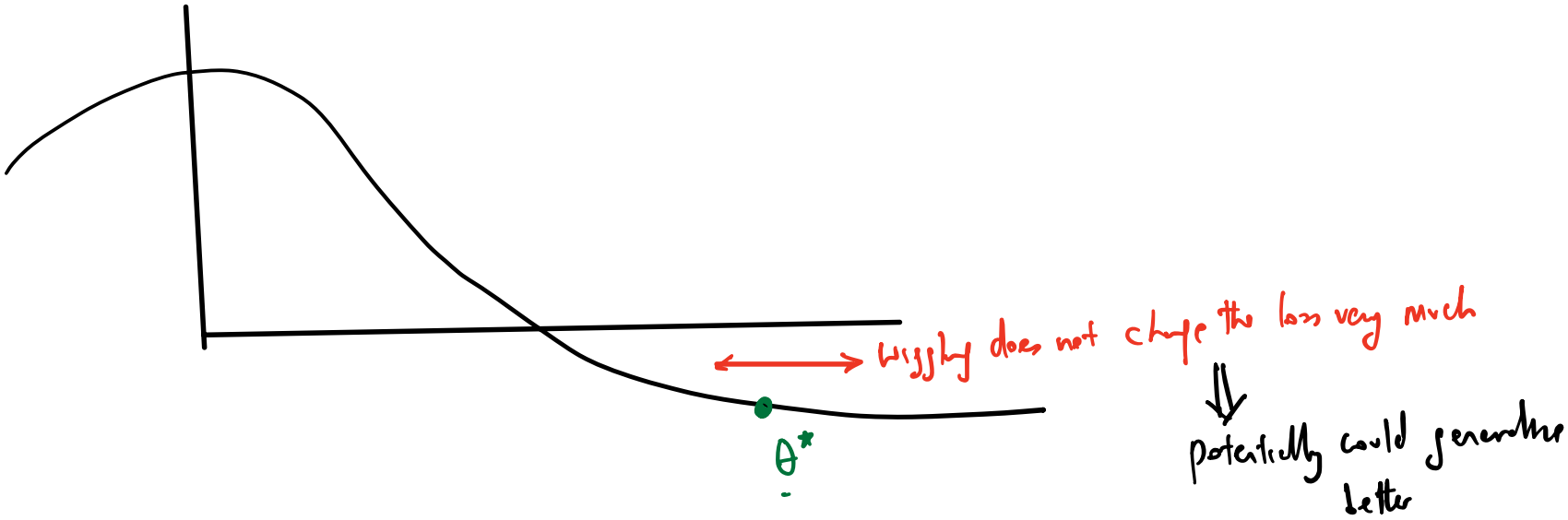
Momentum

```
alpha = 0.9
p = 0
while last_diff > tol:
    cost, g = func(x)
    p = alpha*p - eps*g
    x += p
    last_diff = np.linalg.norm(x - path[-1])
```

Does momentum help with local optima?

Does momentum help with local optima?

What kind of local optima does momentum tend to find?



Nesterov momentum ~ 1983

Nesterov momentum is similar to momentum, except the gradient is calculated at the parameter setting after taking a step along the direction of the momentum.

Initialize $\mathbf{v} = 0$. Then, until stopping criterion is met:

- Update:

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \varepsilon \nabla_\theta J(\theta + \alpha \mathbf{v})$$

change is from $\nabla_\theta(\theta_t)$ to $\nabla_\theta(\theta_t + \alpha v_t)$

- Gradient step:

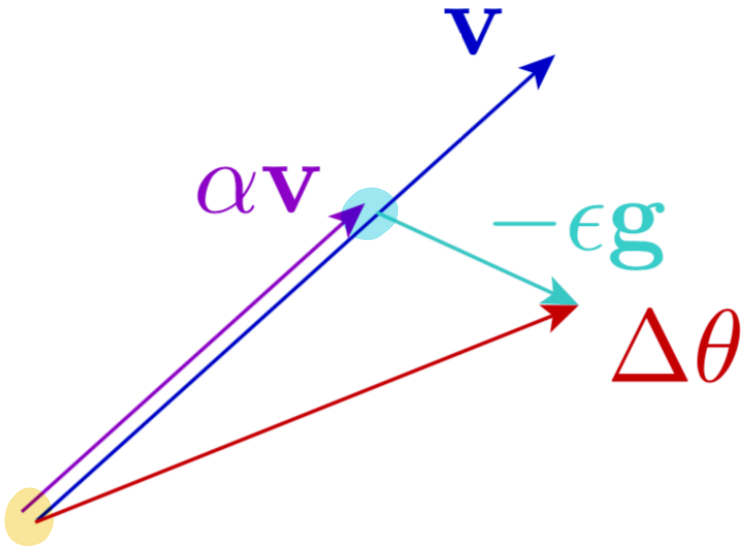
$$\theta \leftarrow \theta + \mathbf{v}$$

Nesterov momentum (cont.)

earlier form $\longrightarrow$ $\underline{v}_{t+1} = \alpha \underline{v}_t - \varepsilon \nabla_{\underline{\theta}} J(\underline{\theta}_t)$

The following image is appropriate for Nesterov momentum: $\underline{v}_{t+1} = \alpha \underline{v}_t - \varepsilon \nabla_{\underline{\theta}} J(\underline{\theta}_t + \alpha \underline{v}_t)$

Both do a step $\alpha \underline{v}_t$ direction



$$\theta_{t+1} = \theta_t + v_{t+1}$$
$$= \theta_t + \alpha v_t - \varepsilon \nabla_{\theta} J(\cdot)$$

Common to both methods

$$\nabla J(\underline{\theta} + \alpha \underline{v}) \approx \nabla J(\underline{\theta}) + \alpha \underline{\underline{H}}\, \underline{v}$$

$\underline{\underline{H}}$ → Hessian

$$\underline{v} \leftarrow \alpha \underline{v} - \varepsilon \nabla J(\underline{\theta} + \alpha \underline{v}) \approx \alpha \underline{v} - \varepsilon \nabla J(\theta) - \varepsilon \alpha \underline{\underline{H}}\, \underline{v}$$

opposes update in high curvature regions

Nesterov momentum

By performing a change of variables with $\tilde{\theta}_{\text{old}} = \theta_{\text{old}} + \alpha \mathbf{v}_{\text{old}}$, it's possible to show that the following is equivalent to Nesterov momentum. (This representation doesn't require evaluating the gradient at $\theta + \alpha \mathbf{v}$.)

- Update:

$$\mathbf{v}_{\text{new}} = \alpha \mathbf{v}_{\text{old}} - \varepsilon \nabla_{\tilde{\theta}_{\text{old}}} J(\tilde{\theta}_{\text{old}})$$

- Gradient step:

$$\tilde{\theta}_{\text{new}} = \tilde{\theta}_{\text{old}} + \mathbf{v}_{\text{new}} + \alpha(\mathbf{v}_{\text{new}} - \mathbf{v}_{\text{old}})$$

- Set $\mathbf{v}_{\text{new}} = \mathbf{v}_{\text{old}}$, $\tilde{\theta}_{\text{new}} = \tilde{\theta}_{\text{old}}$.

Question: Can we do the Nesterov update more efficiently?

$\phi_t = \theta_t + \alpha v_t$, $g_t = \nabla_\theta J(\phi_t)$

$v_{t+1} = \alpha v_t - \epsilon g_t$ ← Nesterov momentum

$\theta_{t+1} = \theta_t + v_{t+1}$

$\phi_{t+1} = \theta_{t+1} + \alpha v_{t+1} = \theta_t + v_{t+1} + \alpha v_{t+1} = \phi_t - \alpha v_t + (1+\alpha) v_{t+1}$

$= \phi_t + v_{t+1} + \alpha [v_{t+1} - v_t]$

$\phi_t - \alpha v_t$

Recall $\theta_{t+1} = \theta_t + \alpha v_t - \epsilon \nabla_\theta J(\theta_t + \alpha v_t)$

ϕ_t

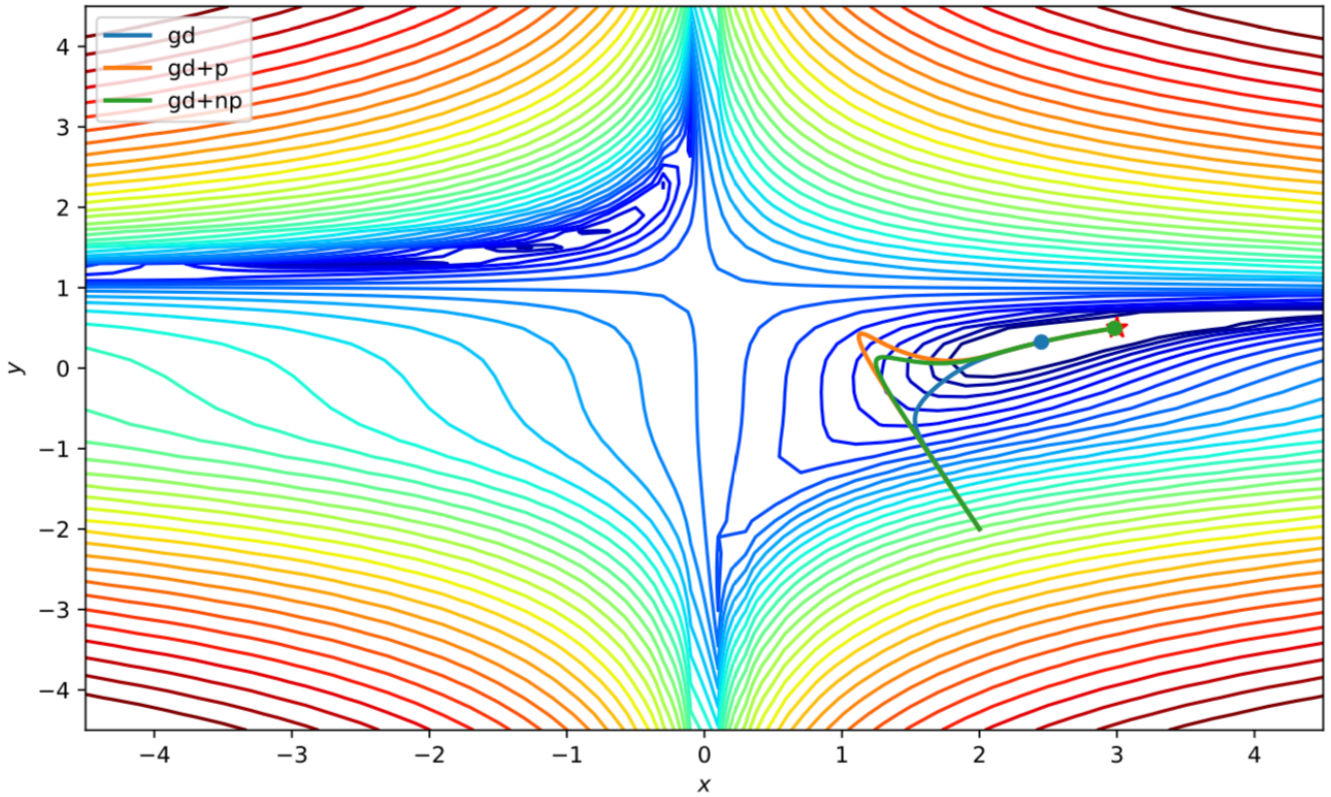
Nesterov momentum

Bottleneck: Maintain $\{\phi_t, v_t\}$ and update ϕ_t directly without doing the partial step of θ_t update

```
alpha = 0.9
p = np.zeros_like(x)
while last_diff > tol:
    cost, g = func(x)
    p_old = p
    p = alpha*p - eps*g
    x += p + alpha*(p-p_old)
    last_diff = np.linalg.norm(x - path[-1])
```

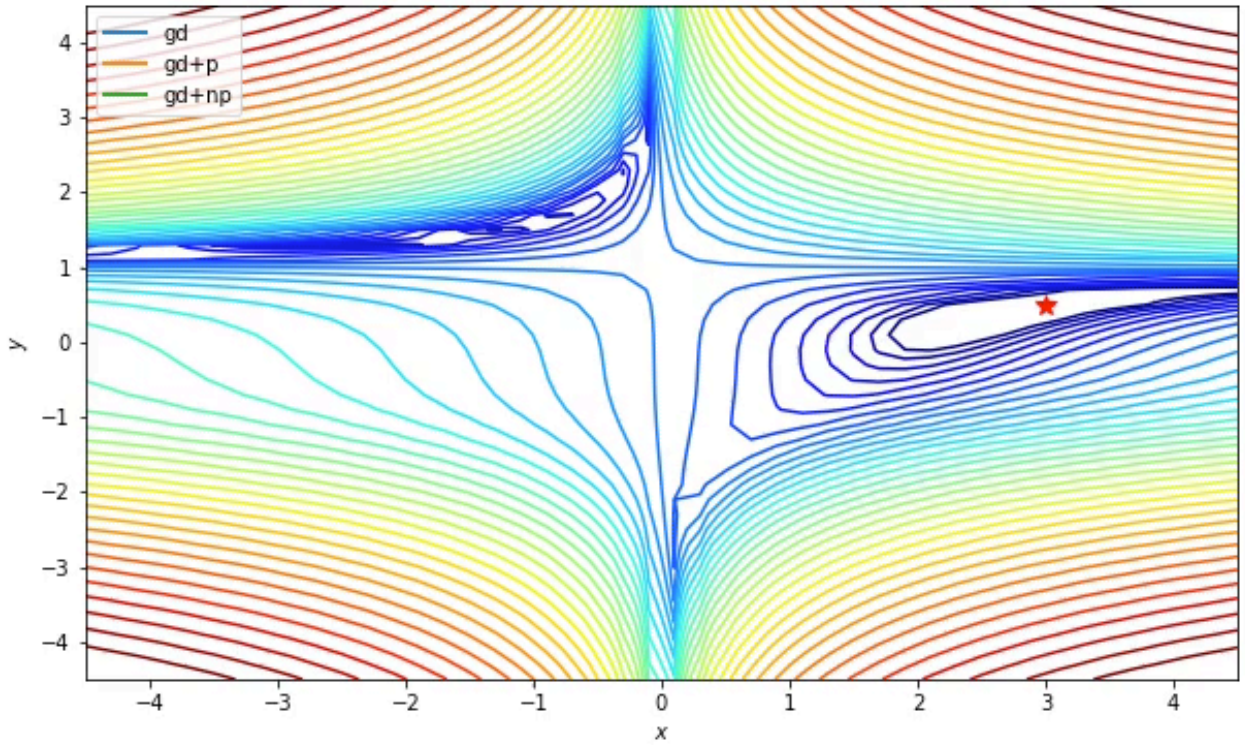
Nesterov momentum (opt 1)

gd+np denotes gradient descent with Nesterov momentum.



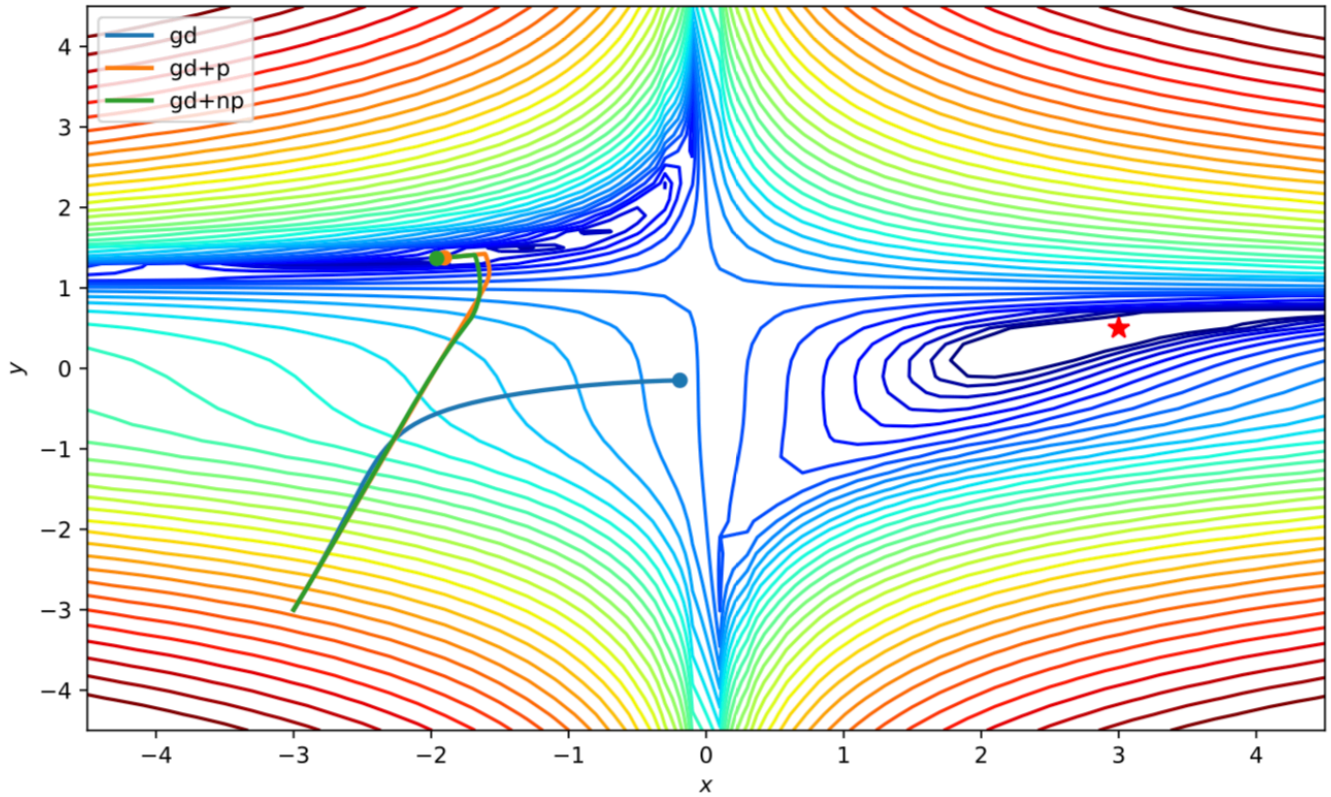
Video: `http://seas.ucla.edu/~kao/opt_anim/1gd_gd+p_gd+np.mp4`

Nesterov momentum



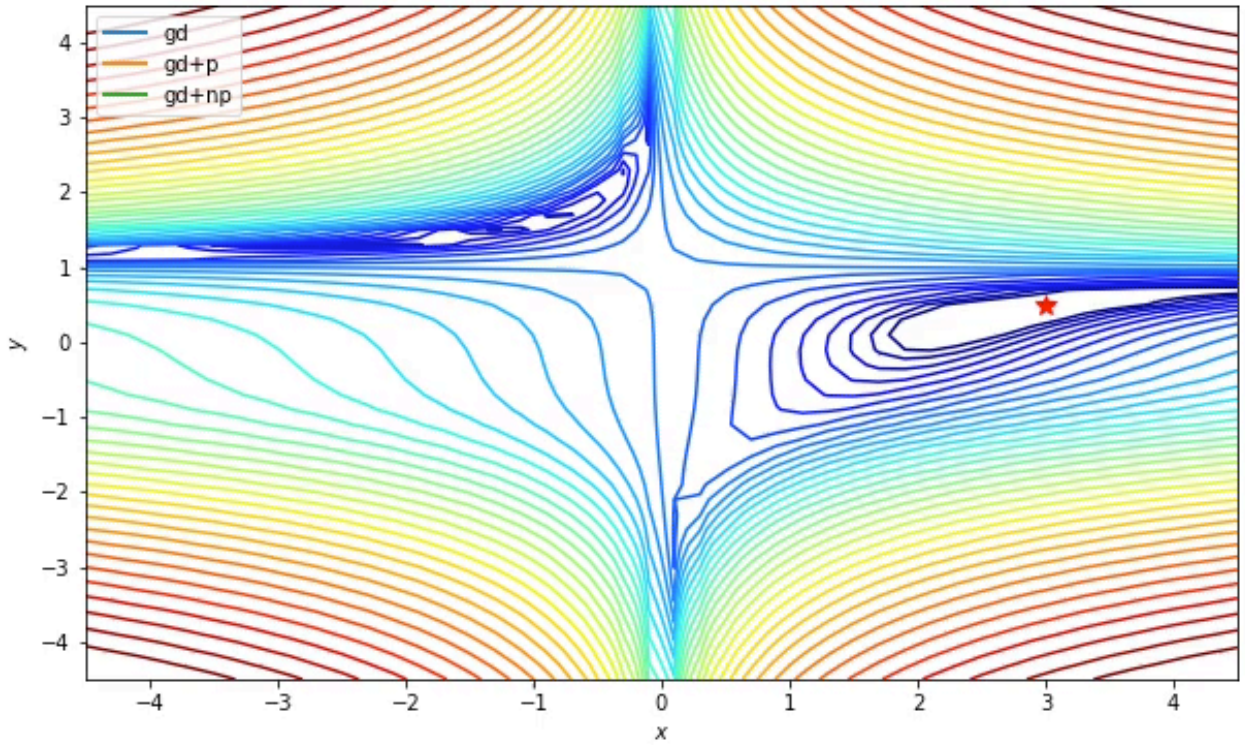
Nesterov momentum (opt 2)

Notice how Nesterov momentum finds the same local minimum as momentum.



Video: `http://seas.ucla.edu/~kao/opt_anim/2gd_gd+p_gd+np.mp4`

Nesterov momentum

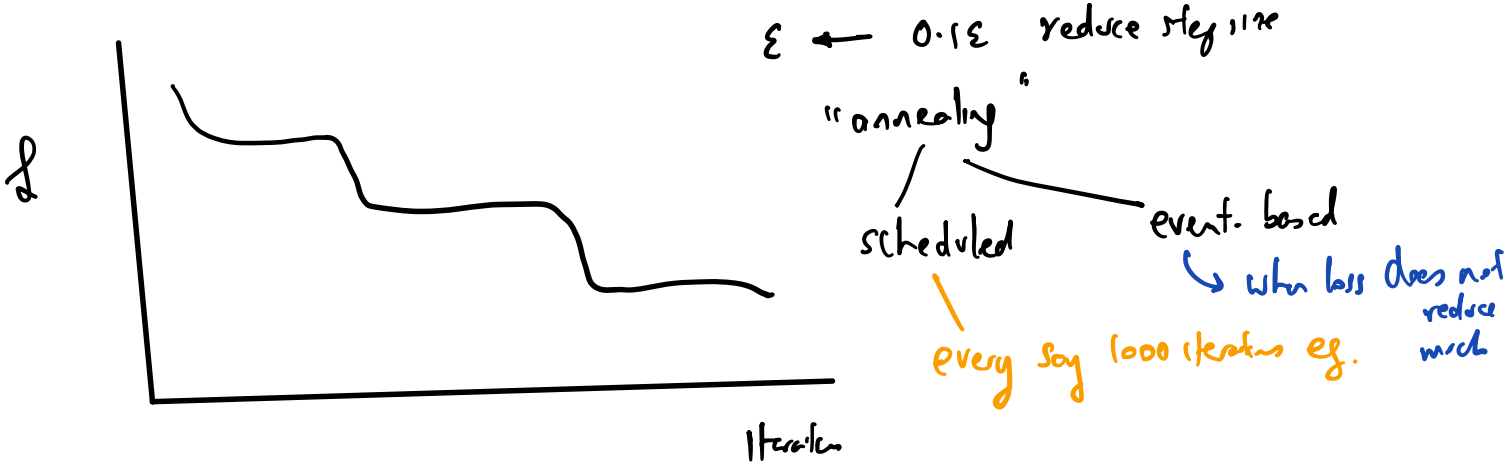


Is there a good way to adapt the learning rule?

Techniques to adapt the learning rate

Choosing ε judiciously can be important for learning. In the beginning, a larger learning rate is typically better, since bigger updates in the parameters may accelerate learning. However, as time goes on, ε may need to be small to be able to make appropriate updates to the parameters. We mentioned before that often times, one applies a decay rule to the learning rate. This is called *annealing*. A common form to anneal the learning rate is to do so manually when the loss plateaus, or to anneal it after a set number of epochs of gradient descent.

Another approach is to update the learning rate based off of the history of gradients.



Adaptive gradient (Adagrad) ← Duchi et al 2011

Adaptive gradient (Adagrad) is a form of stochastic gradient descent where the learning rate is decreased through division by the historical gradient norms. We will let the variable a denote a running sum of squares of gradient norms.

Initialize $\mathbf{a} = 0$. Set ν at a small value to avoid division by zero (e.g., $\nu = 1e-7$). Then, until stopping criterion is met:

- Compute the gradient: $\mathbf{g}$
- Update:

$$\mathbf{a} \leftarrow \mathbf{a} + \mathbf{g} \odot \mathbf{g}$$

- Gradient step:

$$\theta \leftarrow \theta - \frac{\varepsilon}{\sqrt{\mathbf{a}} + \nu} \odot \mathbf{g}$$

$a_{t+1}^{(i)} = a_t^{(i)} + |g_t^{(i)}|^2$

$\underline{\theta} \in \mathbb{R}^n \quad \underline{g} \in \mathbb{R}^n \quad \underline{g} = \begin{bmatrix} g_1 \\ \vdots \\ g_n \end{bmatrix}$

hyperparameter not make division by 0.

$$\begin{bmatrix} \frac{\varepsilon g_1}{\sqrt{a_1 + \nu}} \\ \vdots \\ \frac{\varepsilon g_n}{\sqrt{a_n + \nu}} \end{bmatrix}$$

a_i is larger if g_i is larger

Why?

Smaller steps in direction where we have made large steps in that direction

Hadamard product

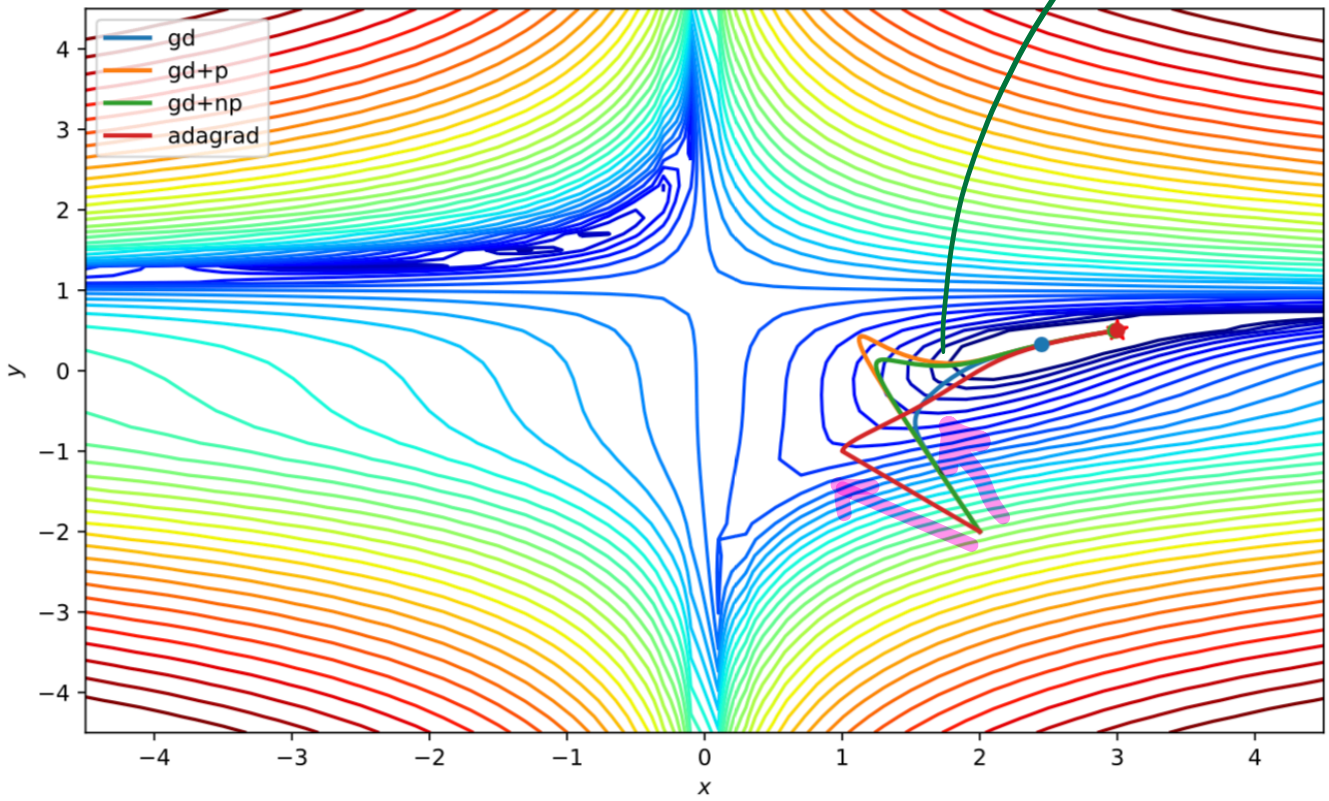
$g \odot g$ = elementwise multiplication

$$= \begin{bmatrix} g_1 \cdot g_1 \\ \vdots \\ g_n \cdot g_n \end{bmatrix} = \begin{bmatrix} g_1^2 \\ \vdots \\ g_n^2 \end{bmatrix}$$

Adagrad

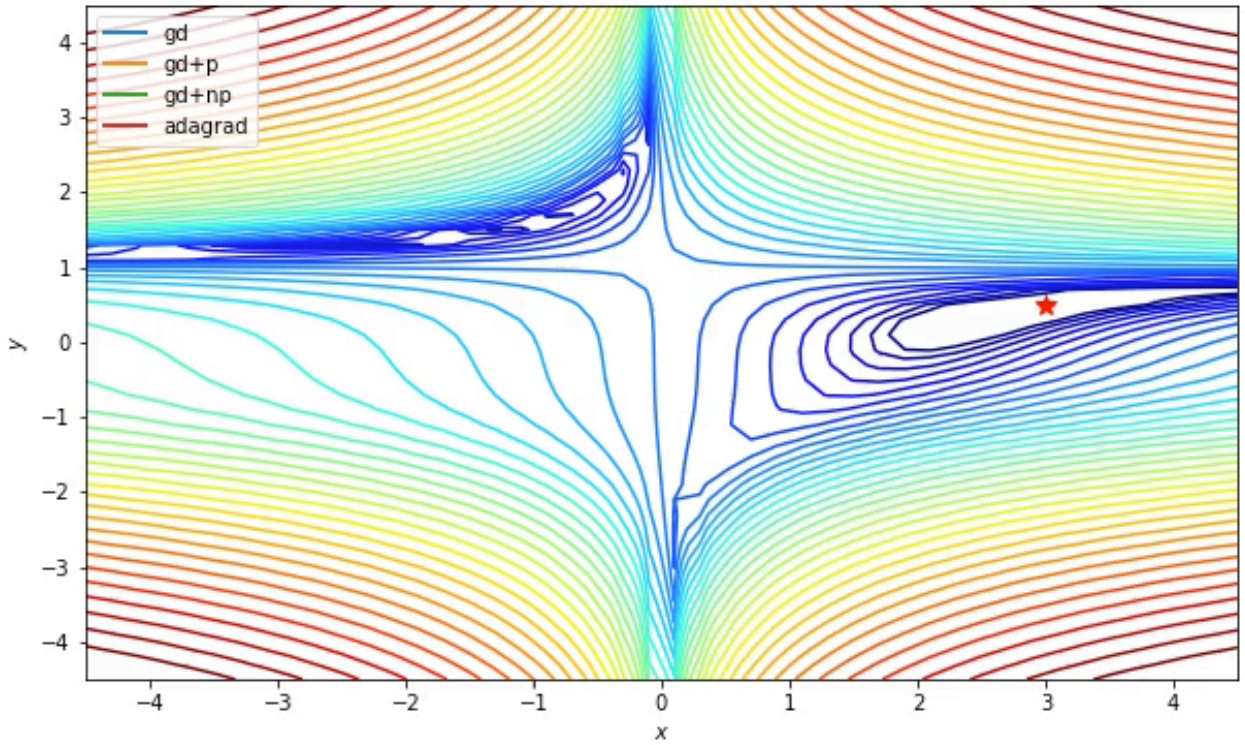
```
a = 0
nu = 1e-7
while last_diff > tol:
    cost, g = func(x)
    a += g*g
    x -= eps * g / (np.sqrt(a) + nu)
    last_diff = np.linalg.norm(x - path[-1])
```

Adagrad (opt 1)



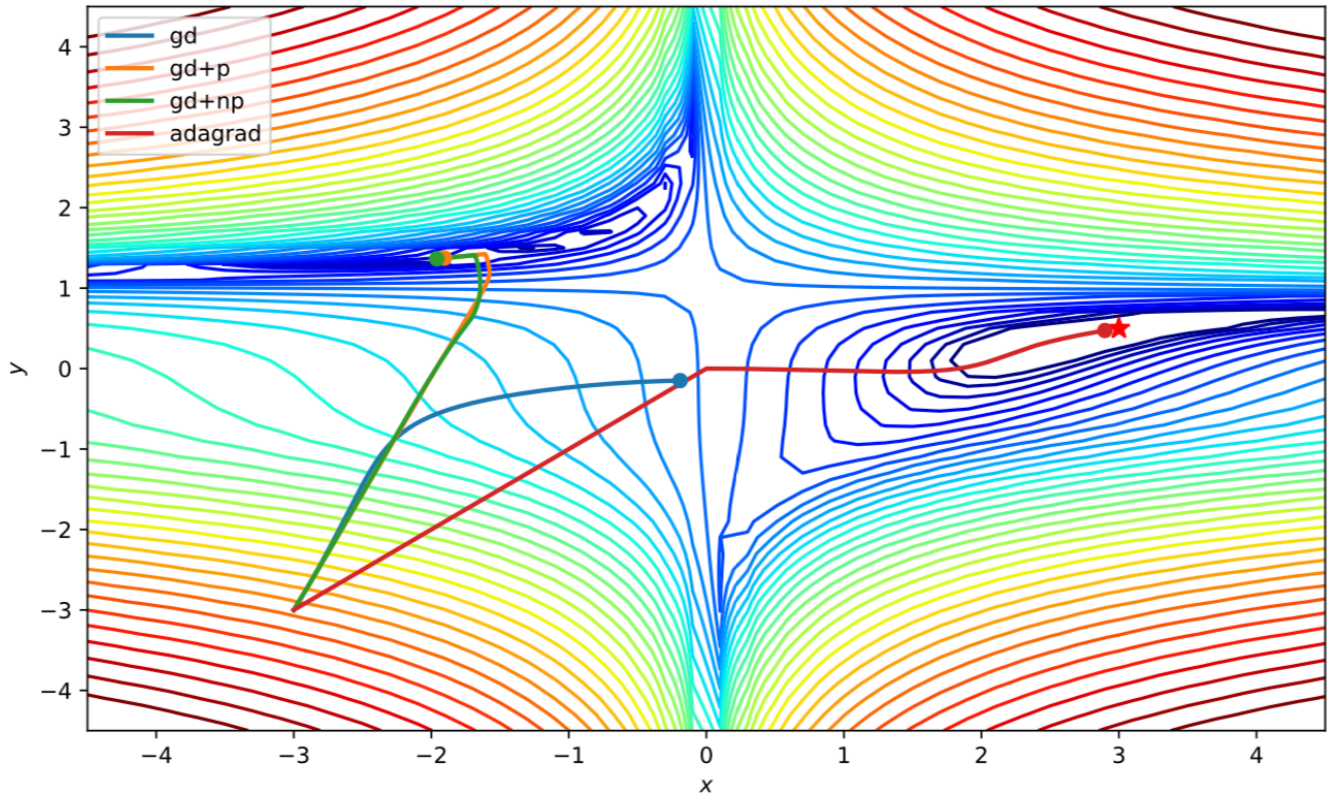
Video: http://seas.ucla.edu/~kao/opt_anim/1gd_gd+p_gd+np_adagrad.mp4

Adagrad



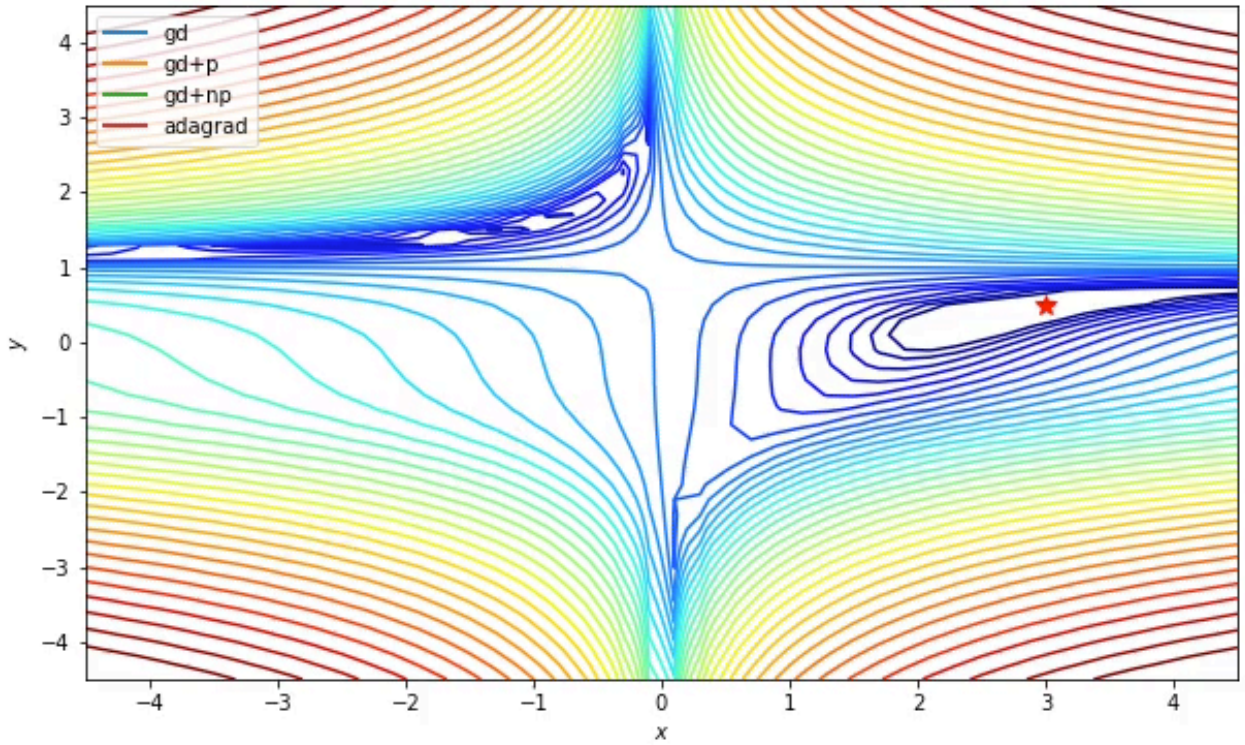
Adagrad (opt 2)

Adagrad proceeds to the global minimum.



Video: `http://seas.ucla.edu/~kao/opt_anim/2gd_gd+p_gd+np_adagrad.mp4`

Adagrad



Adagrad

Is there a problem with adagrad?

- Compute the gradient: $\mathbf{g}$
- Update:

$$\mathbf{a} \leftarrow \mathbf{a} + \mathbf{g} \odot \mathbf{g}$$

$$a_{t+1}^{(i)} = a_t^{(i)} + |g_t^{(i)}|^2$$

- Gradient step:

$$\theta \leftarrow \theta - \frac{\varepsilon}{\sqrt{\mathbf{a}} + \nu} \odot \mathbf{g}$$

CON: a is only increasing

$\Rightarrow$ learning rate is only becoming smaller

RMSProp

RMSProp augments Adagrad by making the gradient accumulator an exponentially weighted moving average.

Initialize $\mathbf{a} = 0$ and set ν to be sufficiently small. Set β to be between 0 and 1 (typically a value like 0.99). Then, until stopping criterion is met:

- Compute the gradient: $\mathbf{g}$
- Update:

$$\mathbf{a} \leftarrow \beta \mathbf{a} + (1-\beta)\mathbf{g} \odot \mathbf{g}$$

← a can become smaller or larger

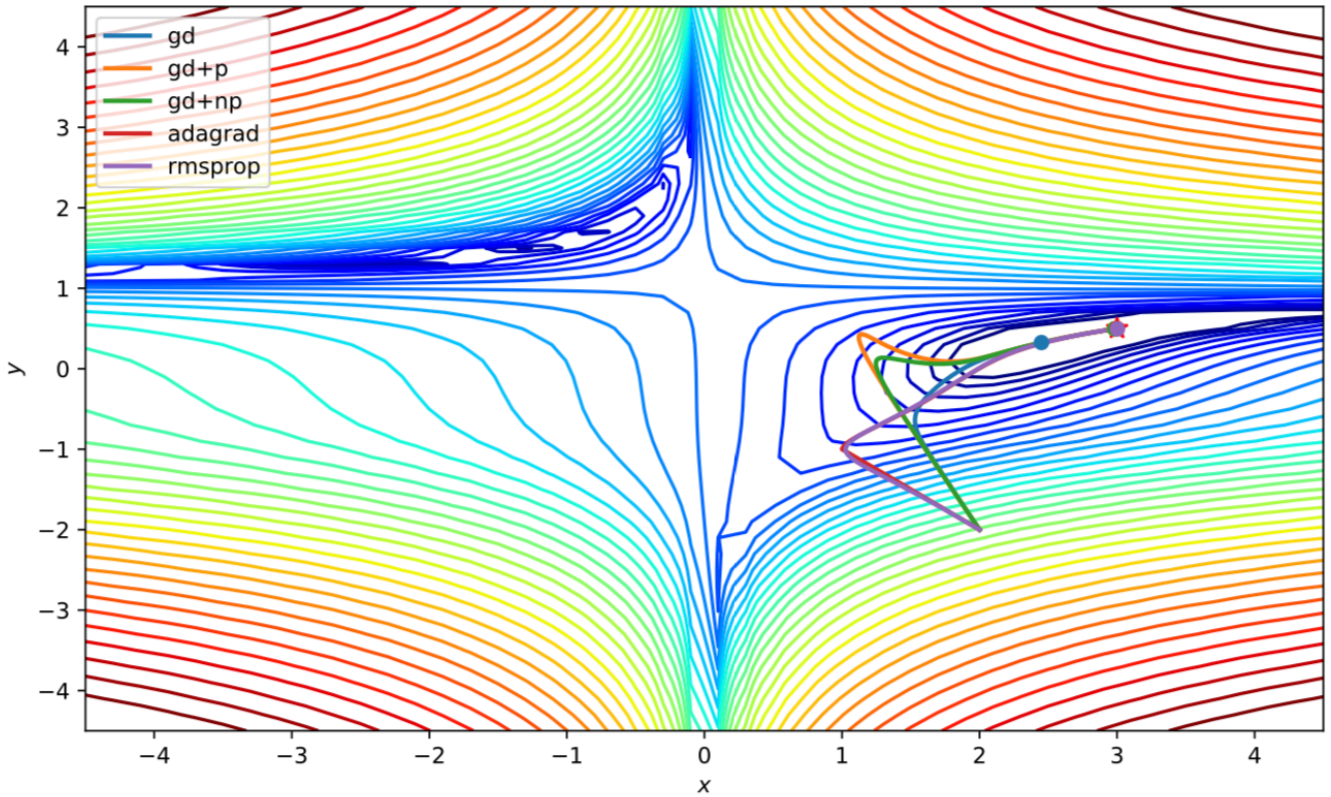
- Gradient step:

$$\theta \leftarrow \theta - \frac{\varepsilon}{\sqrt{\mathbf{a}} + \nu} \odot \mathbf{g}$$

RMSProp

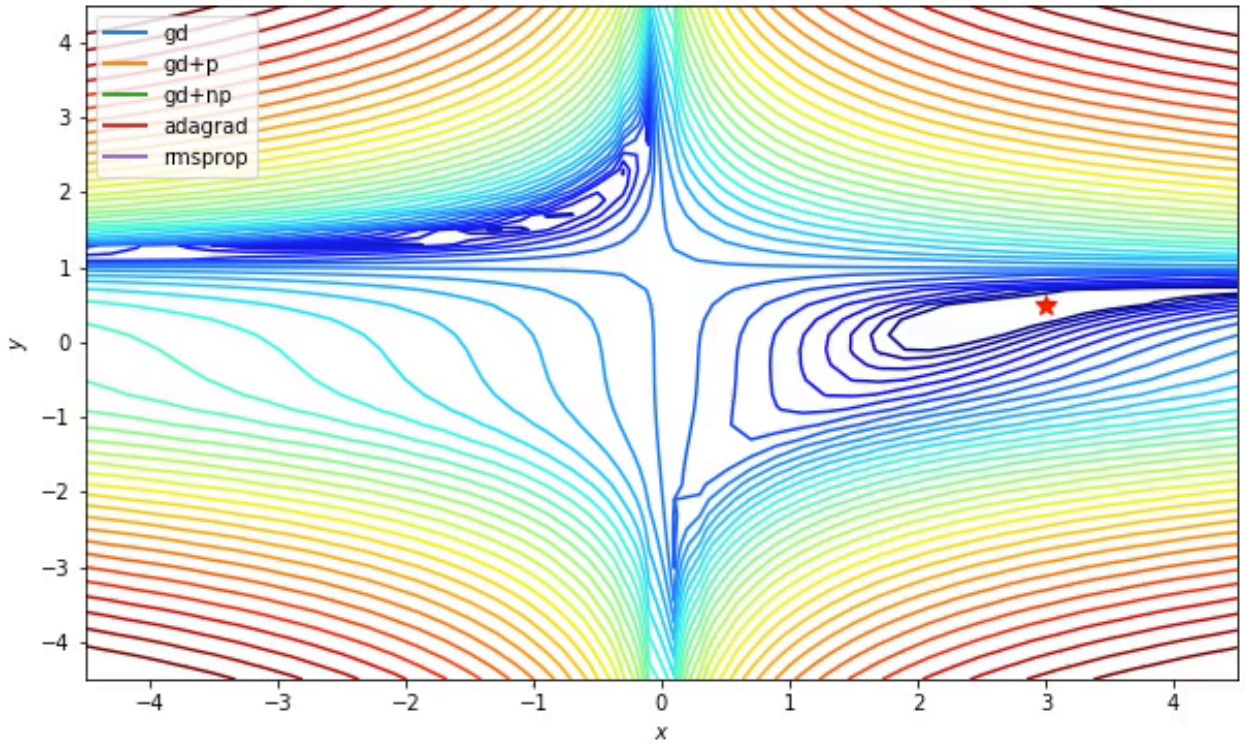
```
a = 0
beta = 0.99
nu = 1e-7
while last_diff > tol:
    cost, g = func(x)
    a = beta * a + (1-beta) * g * g
    x -= eps * g / (np.sqrt(a + nu))
    last_diff = np.linalg.norm(x - path[-1])
```

RMSProp (opt 1)



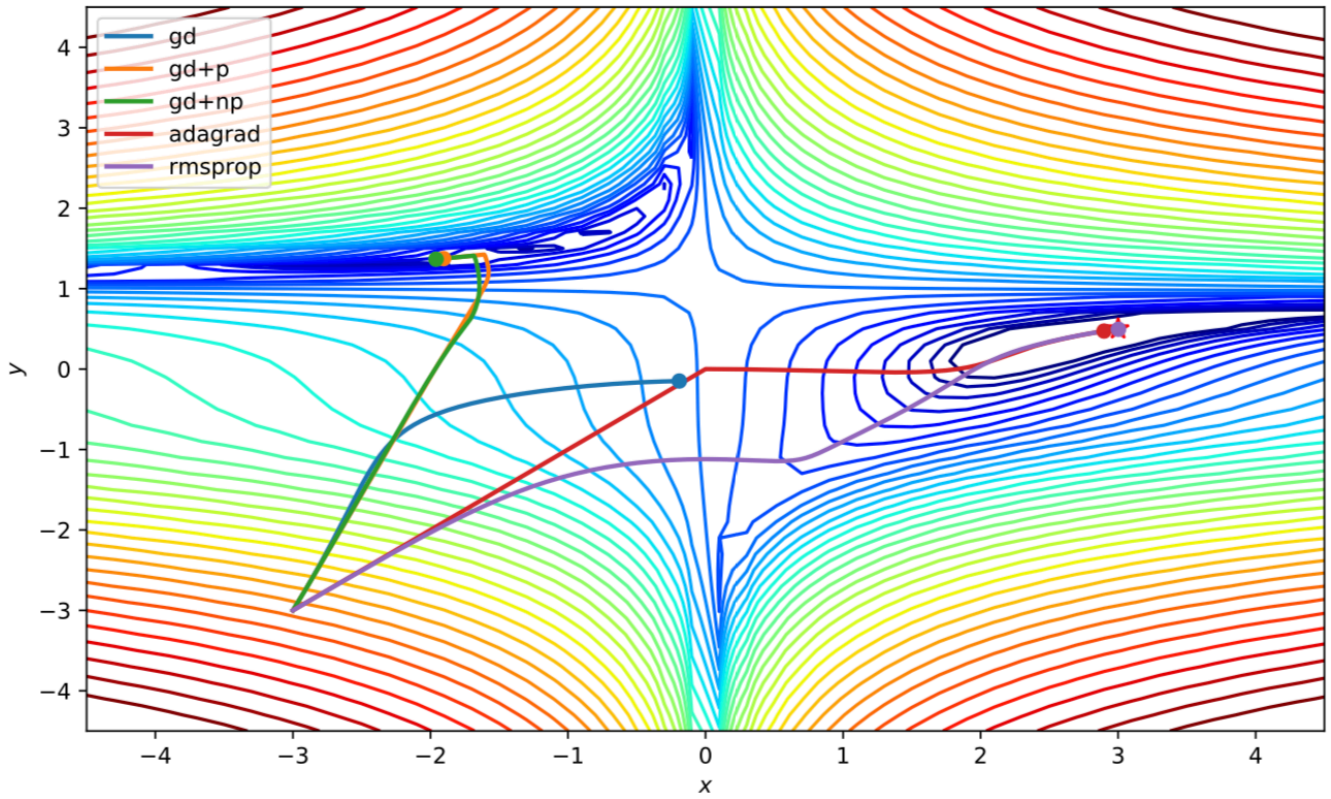
Video: `http://seas.ucla.edu/~kao/opt_anim/1gd_gd+p_gd+np_adagrad_rmsprop.mp4`

RMSProp



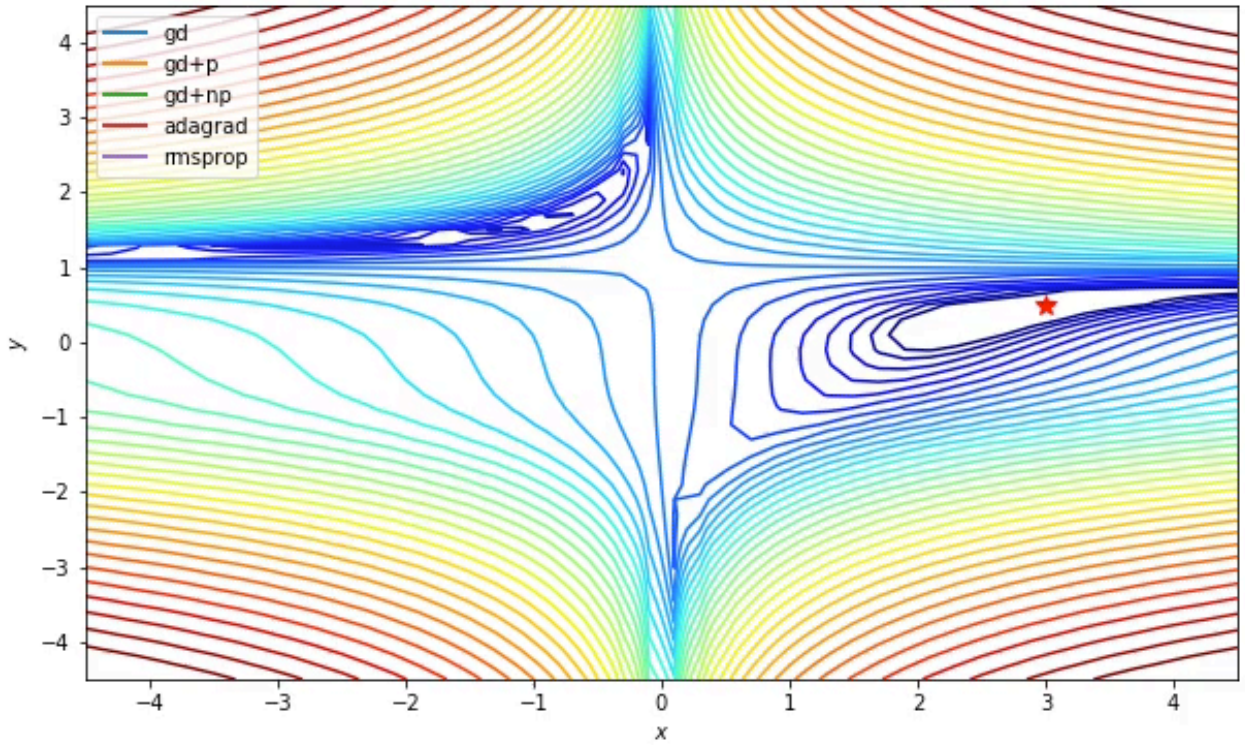
RMSProp (opt 2)

RMSProp proceeds to the global minimum, and in the video you can see it does so more quickly than Adagrad.



Video: `http://seas.ucla.edu/~kao/opt_anim/2gd_gd+p_gd+np_adagrad_rmsprop.mp4`

RMSProp



RMSProp with momentum

RMSProp can be combined with momentum as follows.

Initialize $\mathbf{a} = 0$. Set α, β to be between 0 and 1. Set $\nu = 1e-7$. Until stopping criterion is met:

- Compute gradient: $\mathbf{g}$
- Accumulate gradient:

$$\mathbf{a} \leftarrow \beta \mathbf{a} + (1-\beta)\mathbf{g} \odot \mathbf{g}$$

- Momentum:

$$\mathbf{v} \leftarrow \alpha \mathbf{v} - \frac{\varepsilon}{\sqrt{\mathbf{a}+\nu}} \odot \mathbf{g}$$

- Gradient step:

$$\theta \leftarrow \theta + \mathbf{v}$$

It is also possible to RMSProp with Nesterov momentum.

Skip because we will do Adam etc

Adam

Adaptive moments without bias correction ~ 2015

The adaptive moments optimizer (Adam) is one of the most commonly used (and robust to e.g., hyperparameter choice) optimizers. Adam is composed of a momentum-like step, followed by an Adagrad/RMSProp-like step. For intuition, we first present Adam without a bias correction step.

Adam with no bias correction

Initialize $\mathbf{v} = 0$ as the "first moment", and $\mathbf{a} = 0$ as the "second moment." Set β_1 and β_2 to be between 0 and 1. (Suggested defaults are $\beta_1 = 0.9$ and $\beta_2 = 0.999$.) Initialize ν to be sufficiently small. Until stopping criterion is met:

ordinary momentum → First moment g

2^{nd} moment g^2

ADAM

- Compute gradient: $\mathbf{g}$
- First moment update (momentum-like):

$$\mathbf{v} \leftarrow \beta_1 \mathbf{v} + (1 - \beta_1)\mathbf{g}$$

- Second moment update (gradient normalization):

$$\mathbf{a} \leftarrow \beta_2 \mathbf{a} + (1 - \beta_2)\mathbf{g} \odot \mathbf{g}$$

- Gradient step:

$$\theta \leftarrow \theta - \frac{\varepsilon}{\sqrt{\mathbf{a}} + \nu} \odot \mathbf{v}$$

Adaptive moments (Adam)

Adam incorporates a bias correction on the moments. The intuition for the bias correction is to account for initialization; these bias corrections amplify the second moments, so that extremely large steps are not taken at the start of the optimization.

Adam (cont.)

Initialize $\mathbf{v} = 0$ as the "first moment", and $\mathbf{a} = 0$ as the "second moment." Set β_1 and β_2 to be between 0 and 1. (Suggested defaults are $\beta_1 = 0.9$ and $\beta_2 = 0.999$.) Initialize ν to be sufficiently small. Initialize $t = 0$. Until stopping criterion is met:

- Compute gradient: $\mathbf{g}$
- Time update: $t \leftarrow t + 1$
- First moment update (momentum-like):

$$\mathbf{v} \leftarrow \beta_1 \mathbf{v} + (1 - \beta_1)\mathbf{g}$$

- Second moment update (gradient normalization):

$$\mathbf{a} \leftarrow \beta_2 \mathbf{a} + (1 - \beta_2)\mathbf{g} \odot \mathbf{g}$$

- Bias correction in moments:

$$\tilde{\mathbf{v}} = \frac{1}{1 - \beta_1^t}\mathbf{v}$$

$$\tilde{\mathbf{a}} = \frac{1}{1 - \beta_2^t}\mathbf{a}$$

- Gradient step:

$$\theta \leftarrow \theta - \frac{\varepsilon}{\sqrt{\tilde{\mathbf{a}}} + \nu} \odot \tilde{\mathbf{v}}$$

$\beta_1, \beta_2 \in [0, 1]$

t iteration count

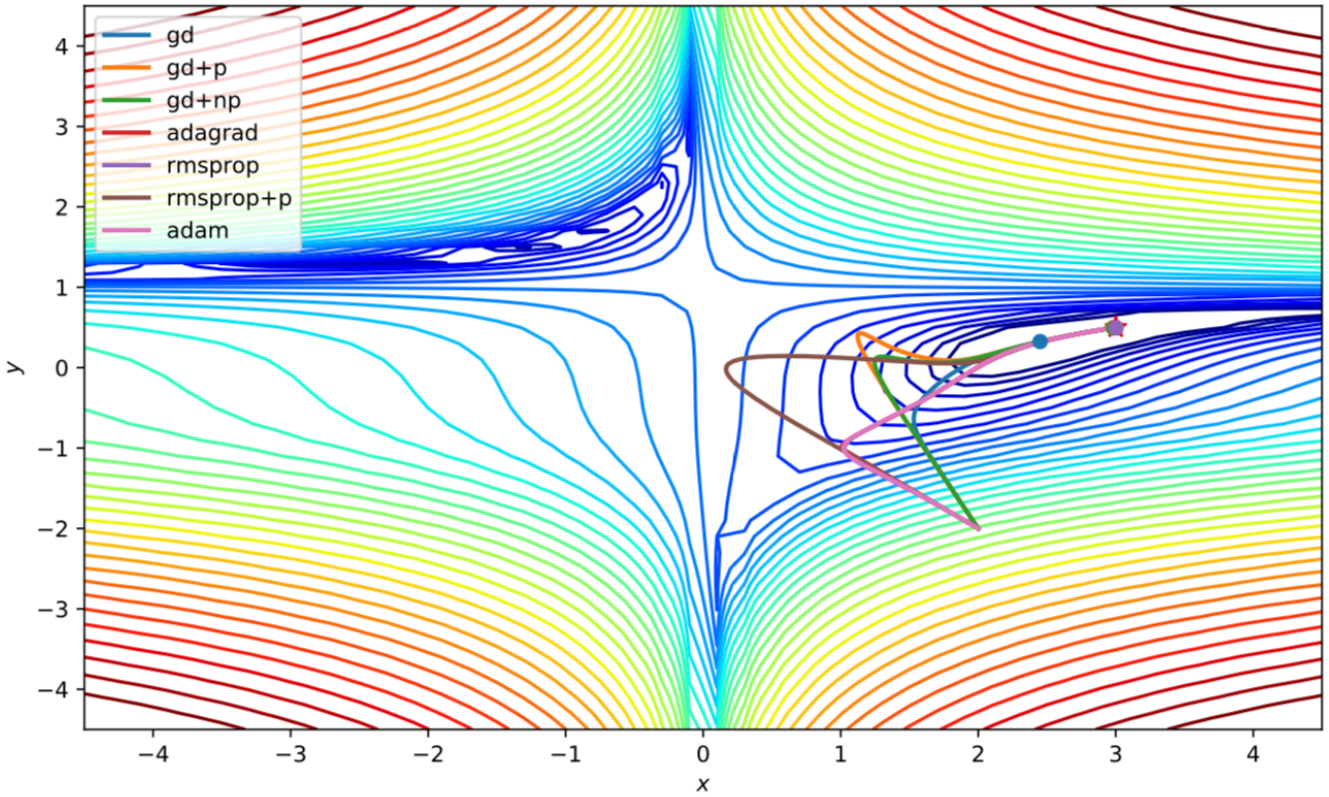
A resource on bias correction: https://www.coursera.org/lecture/deep-neural-network/bias-correction-in-exponentially-weighted-averages-XjuhD

Adam

```
a = 0
p = 0
beta1 = 0.9
beta2 = 0.99
nu = 1e-7
t = 0
while last_diff > tol:
    cost, g = func(x)
    t += 1
    p = beta1 * p + (1-beta1) * g
    a = beta2 * a + (1-beta2) * g * g
    p_u = p / (1 - beta1**t)
    a_u = a / (1 - beta2**t)
    x -= eps * p_u / (np.sqrt(a_u) + nu)
    last_diff = np.linalg.norm(x - path[-1])
```

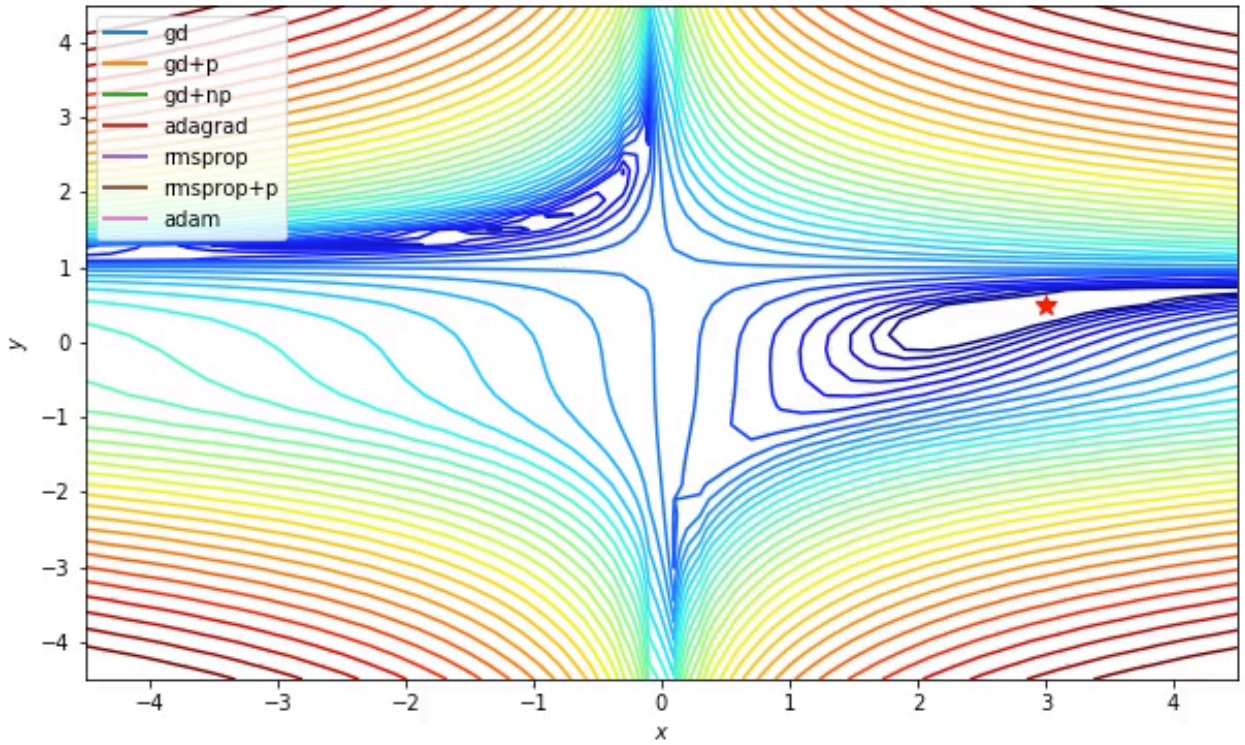
Adam (opt 1)

In both example optimizations, Adam is just slightly slower than RMSProp.

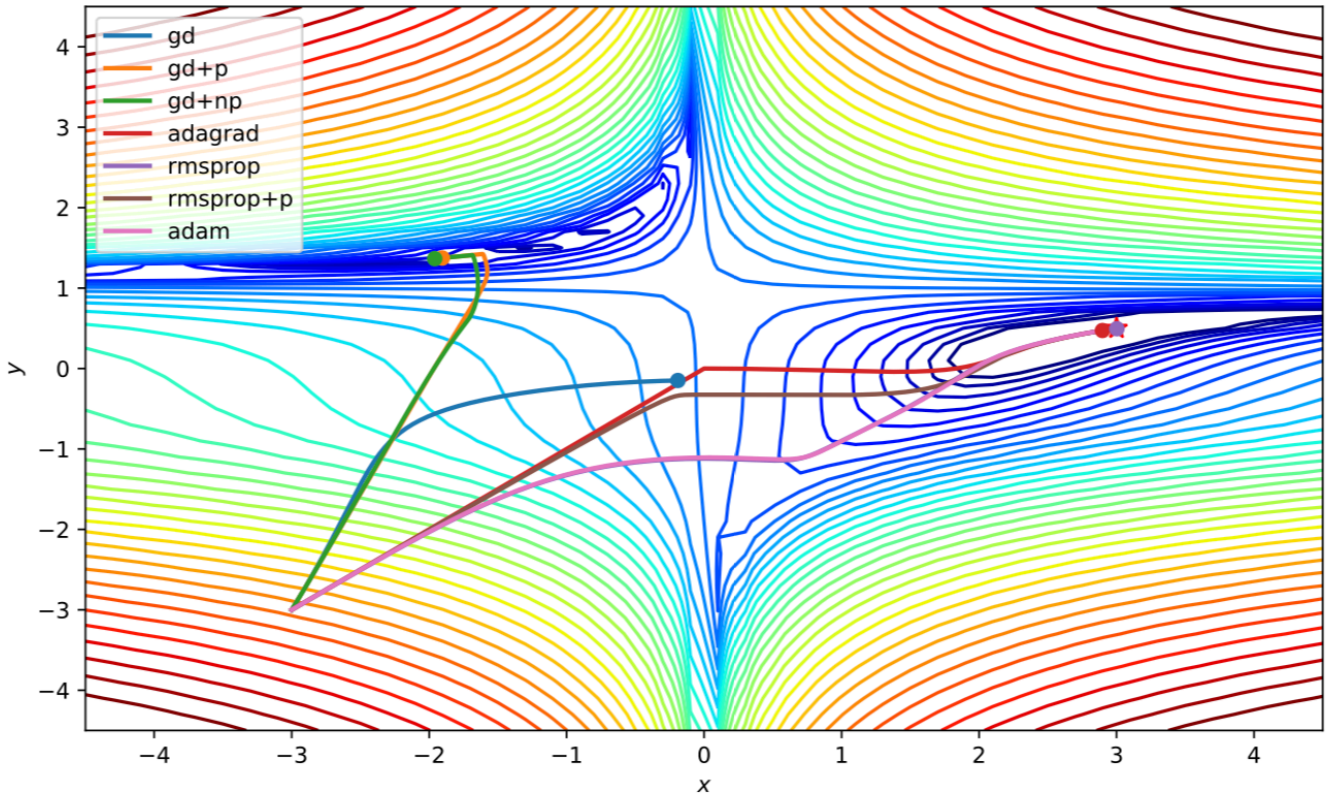


Video: `http://seas.ucla.edu/~kao/opt_anim/1gd_gd+p_gd+np_adagrad_rmsprop+p_adam.mp4`

Adam

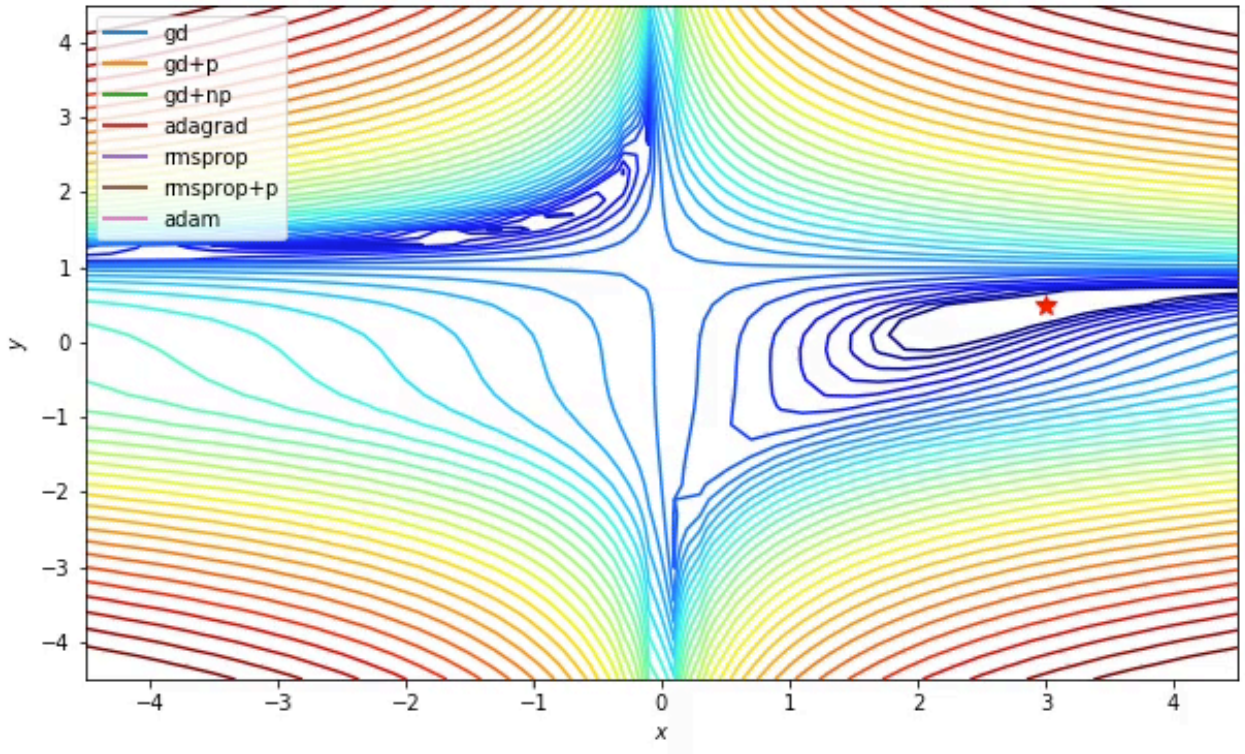

gd
gd+p
gd+np
adagrad
rmsprop
rmsprop+p
adam
y
x
4
3
2
1
0
−1
−2
−3
−4
−4
−3
−2
−1
0
1
2
3
4

Adam (opt 2)



Video: `http://seas.ucla.edu/~kao/opt_anim/2gd_gd+p_gd+np_adagrad_rmsprop+p_adam.mp4`

Adam


gd
gd+p
gd+np
adagrad
rmsprop
rmsprop+p
adam
4
3
2
1
0
−1
−2
−3
−4
y
−4
−3
−2
−1
0
1
2
3
4
x

AdamW

Published as a conference paper at ICLR 2019

DECOUPLED WEIGHT DECAY REGULARIZATION

Ilya Loshchilov & Frank Hutter
University of Freiburg
Freiburg, Germany,
ilya.loshchilov@gmail.com, fh@cs.uni-freiburg.de

ABSTRACT

L_2 regularization and weight decay regularization are equivalent for standard stochastic gradient descent (when rescaled by the learning rate), but as we demonstrate this is *not* the case for adaptive gradient algorithms, such as Adam. While common implementations of these algorithms employ L_2 regularization (often calling it "weight decay" in what may be misleading due to the inequivalence we expose), we propose a simple modification to recover the original formulation of weight decay regularization by *decoupling* the weight decay from the optimization steps taken w.r.t. the loss function. We provide empirical evidence that our proposed modification (i) decouples the optimal choice of weight decay factor from the setting of the learning rate for both standard SGD and Adam and (ii) substantially improves Adam's generalization performance, allowing it to compete with SGD with momentum on image classification datasets (on which it was previously typically outperformed by the latter). Our proposed decoupled weight decay has already been adopted by many researchers, and the community has implemented it in TensorFlow and PyTorch; the complete source code for our experiments is available at https://github.com/loshchil/AdamW-and-SGDW

https://openreview.net/pdf?id=Bkg6RiCqY7

AdamW → addresses a small "bug" in applying ADAM to l_2-regularized

What is AdamW?

Issue: If we do ADAM with then effectively the regularization is different in different directions → not the intent.

- When combining Adam with L2 regularization (or "weight decay"), the first step looks like this:
  - Compute gradient: $\mathbf{g}$

$$\mathbf{g} = \nabla_\theta J(\theta) + \lambda\theta$$

$$\tilde{J}(\underline{\theta}) = J(\underline{\theta}) + \frac{1}{2}\lambda\|\underline{\theta}\|^2$$

$$\underline{g} = \nabla\tilde{J}(\underline{\theta}) = \nabla J(\underline{\theta}) + \lambda\underline{\theta}$$

- There's a subtle thing here: the L2 term appears in the gradient and now all the additional steps of Adam also scale these L2 gradients. This means that different parameters experience different effective L2 regularization. This is usually **not** what is intended when we L2 regularize.

- AdamW decouples the L2 regularization terms from the additional steps of Adam. It therefore computes the update:

$$\theta \leftarrow \theta - \alpha\text{AdamUpdate}(\mathbf{g}) - \alpha\lambda\theta$$

- Empirically, this performs better and you should probably be using it instead of Adam.

AdamW

Algorithm 2 Adam with L_2 regularization and Adam with decoupled weight decay (AdamW)

given $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, \lambda \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{\theta}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, second moment vector $\boldsymbol{v}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{\theta}_{t-1}) \leftarrow \text{SelectBatch}(\boldsymbol{\theta}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{\theta}_{t-1}) + \lambda \boldsymbol{\theta}_{t-1}$
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1)\boldsymbol{g}_t$ ▷ here and below all operations are element-wise
  $\boldsymbol{v}_t \leftarrow \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2)\boldsymbol{g}_t^2$
  $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t / (1 - \beta_1^t)$ ▷ β_1 is taken to the power of t
  $\hat{\boldsymbol{v}}_t \leftarrow \boldsymbol{v}_t / (1 - \beta_2^t)$ ▷ β_2 is taken to the power of t
  $\eta_t \leftarrow \text{SetScheduleMultiplier}(t)$ ▷ can be fixed, decay, or also be used for warm restarts
  $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \eta_t \left(\alpha \hat{\boldsymbol{m}}_t / (\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon) + \lambda \boldsymbol{\theta}_{t-1} \right)$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{\theta}_t$

https://openreview.net/pdf?id=Bkg6RiCqY7

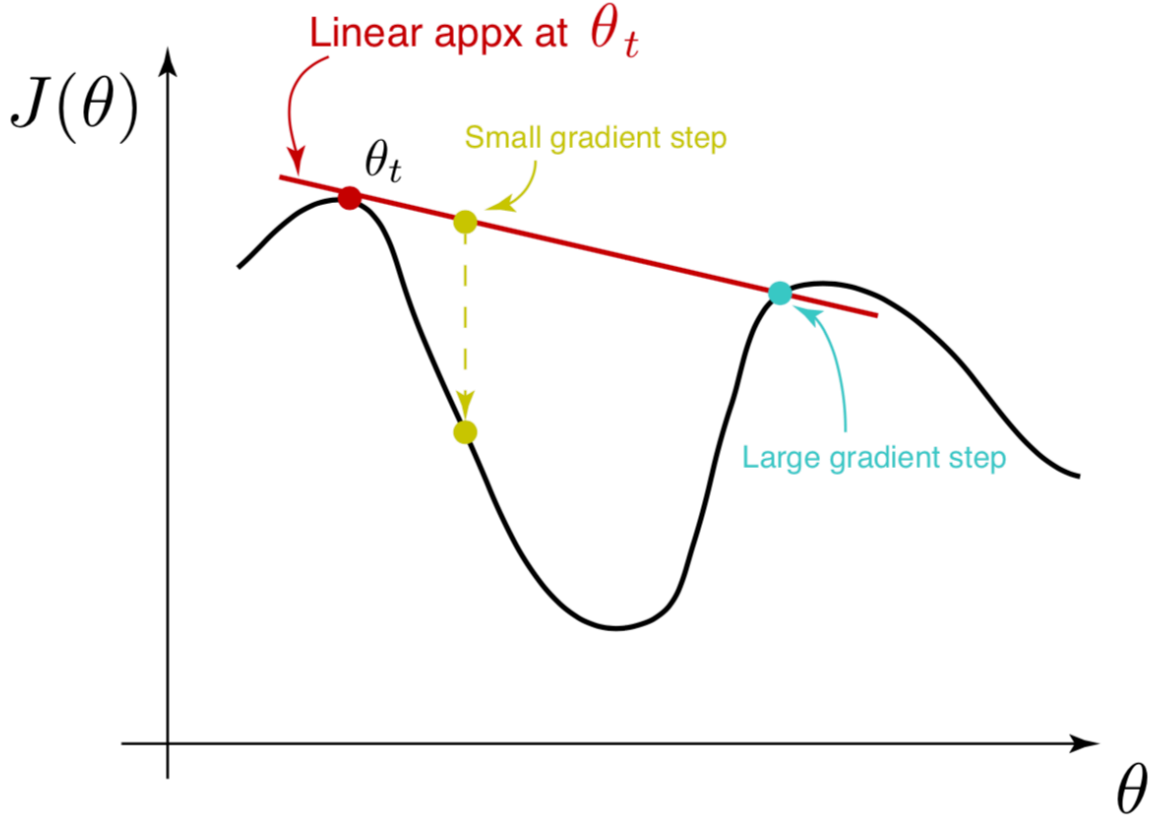
1st order: use $g = \nabla_\theta J(\theta)$

2nd order $H = \nabla_\theta^2 J(\theta)$ → Hessian

First order vs second order methods

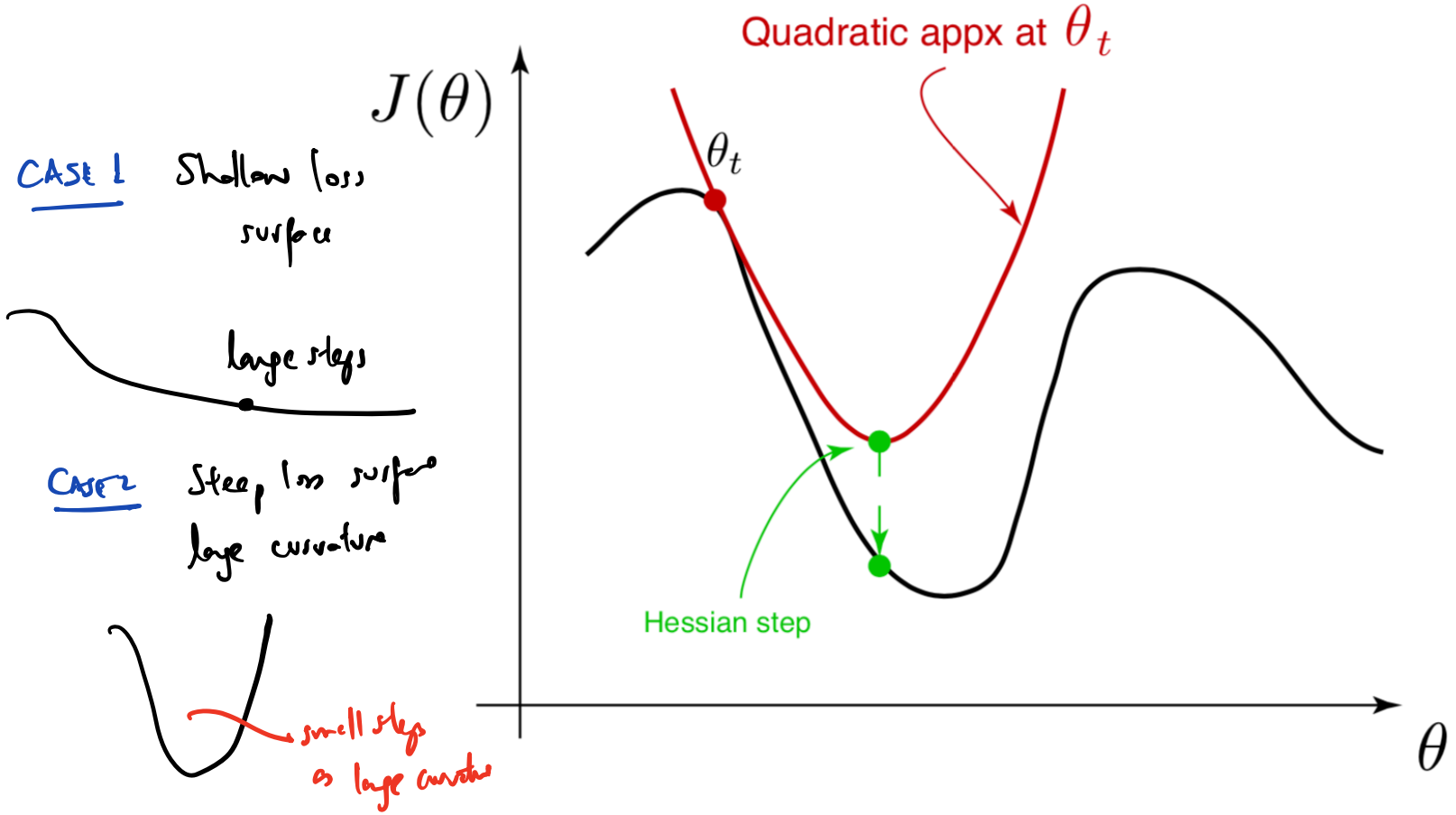
SGD, and optimizers used to augment the learning rate (Adagrad, RMSProp, and Adam), are all *first order* methods and have the learning rate ε as a hyperparameter.

- First-order refers to the fact that we only use the first derivative, i.e., the gradient, and take linear steps along the gradient.
- The following picture of a first order method is appropriate.



First order vs second order methods (cont)

It is possible to also use the *curvature* of the cost function to know how to take steps. These are called second-order methods, because they use the second derivative (or Hessian) to assess the curvature and thus take appropriate sized steps in each dimension. See following picture for intuition:



Second order optimization

The most widely used second order method is Newton's method. To arrive at it, consider the Taylor series expansion of $J(\theta)$ around θ_0 up to the second order terms, i.e.,

$$J(\theta) \approx J(\theta_0) + (\theta - \theta_0)^T \nabla_\theta J(\theta_0) + \frac{1}{2}(\theta - \theta_0)^T \mathbf{H}(\theta - \theta_0)$$

If this were just a second order function, we could minimize it by taking its derivative and setting it to zero:

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta J(\theta_0) + \mathbf{H}(\theta - \theta_0) \\ &= \mathbf{0} \end{aligned}$$

This results in the Newton step,

$$\theta = \theta_0 - \mathbf{H}^{-1} \nabla_\theta J(\theta_0)$$

adapting step size to curvature

If $\mathbf{H}$ is small large step size

If $\mathbf{H}$ is large small step

If the function is quadratic, this step takes us to the minimum. If not, this approximates the function as quadratic at θ_0 and goes to the minimum of the quadratic approximation.

Does this form of step make intuitive sense?

Newton's method

Newton's method, by using the curvature information in the Hessian, does not require a learning rate.

Until stopping criterion is met:

- Compute gradient: $\mathbf{g}$
- Compute Hessian: $\mathbf{H}$
- Gradient step:

$$\theta \leftarrow \theta - \mathbf{H}^{-1}\mathbf{g}$$

Newton's method (cont.)

A few notes about Newton's method:

- When the Hessian has negative eigenvalues, then steps along the corresponding eigenvectors are gradient ascent steps. To counteract this, it is possible to regularize the Hessian, so that the updates become:

$$\theta \leftarrow \theta - (\mathbf{H} + \alpha \mathbf{I})^{-1} \nabla_\theta J(\theta_0)$$

  As α becomes larger, this turns into first order gradient descent with learning rate $1/\alpha$.

- Newton's method requires, at every iteration, calculating and then inverting the Hessian. If the network has N parameters, then inverting the Hessian is $\mathcal{O}(N^3)$. This renders Newton's method impractical for many types of deep neural networks.

Quasi-Newton methods

To get around the problem of having to compute and invert the Hessian, quasi-Newton methods are often used. Amongst the most well-known is the BFGS (Broyden Fletcher Goldfarb Shanno) update.

- The idea is that instead of computing and inverting the Hessian at each iteration, the inverse Hessian $\mathbf{H}_0^{-1}$ is initialized at some value, and it is recursively updated via:

$$\mathbf{H}_k^{-1} \leftarrow \left(\mathbf{I} - \frac{\mathbf{s}\mathbf{y}^T}{\mathbf{y}^T\mathbf{s}}\right)\mathbf{H}_{k-1}^{-1}\left(\mathbf{I} - \frac{\mathbf{y}\mathbf{s}^T}{\mathbf{y}^T\mathbf{s}}\right) + \frac{\mathbf{s}\mathbf{s}^T}{\mathbf{y}^T\mathbf{s}}$$

for

$$\mathbf{s} = \theta_k - \theta_{k-1} \quad \text{and} \quad \mathbf{y} = \nabla J(\theta_k) - \nabla J(\theta_{k-1})$$

- The proof of this result is beyond the scope of this class; if you'd like to learn more about this (and about optimization in general), consider taking ECE 236C.

Quasi-Newton methods (cont.)

- An important aspect of this update is that the inverse of any Hessian can be reconstructed from the sequence of $\mathbf{s}_k$, $\mathbf{y}_k$, and the initial $\mathbf{H}_0^{-1}$. Thus a recurrence relationship can be written to calculate $\mathbf{H}_k^{-1}\mathbf{x}$ without explicitly having to calculate $\mathbf{H}_k^{-1}$. However, it does require iterating over $i = 0, \ldots, k$ examples.
- A way around this is to use limited memory BFGS (L-BFGS), where you calculate the inverse Hessian using just the last m examples assuming $\mathbf{H}_{k-m}^{-1}$ is some $\mathbf{H}_0^{-1}$ (e.g., it could be the identity matrix).
- Quasi-Newton methods usually require a full batch (or very large minibatches) since errors in estimating the inverse Hessian can result in poor steps.

Conjugate gradient methods

CG methods are also beyond the scope of this class, but we bring it up here in case helpful to look into further. Again, ECE 236C is recommended if you'd like to learn more about these techniques.

- CG methods find search directions that are *conjugate* with respect to the Hessian, i.e., that $\mathbf{g}_k^T \mathbf{H} g_{k-1} = 0$.
- It turns out that these derivatives can be calculated iteratively through a recurrence relation.
- Implementations of "Hessian-free" CG methods have been demonstrated to converge well (e.g., Martens et al., ICML 2011).

Challenges in gradient descent

- **Exploding gradients.**
  Sometimes the cost function can have "cliffs" whereby small changes in the parameters can drastically change the cost function. (This usually happens if parameters are repeatedly multiplied together, as in recurrent neural networks.) Because the gradient at a cliff is large, an update can result in going to a completely different parameter space. This can be ameliorated via gradient clipping, which upper bounds the maximum gradient norm.

Challenges in gradient descent

- **Vanishing gradients.**
  Like in exploding gradients, repeated multiplication of a matrix $\mathbf{W}$ can cause vanishing gradients. Say that each time step can be thought of as a layer of a feedforward network where each layer has connectivity $\mathbf{W}$ to the next layer. By layer t, there have been $\mathbf{W}^t$ multiplications. If $\mathbf{W} = \mathbf{U}\Lambda\mathbf{U}^{-1}$ is its eigendecomposition, then $\mathbf{W}^t = \mathbf{U}\Lambda^t\mathbf{U}^{-1}$, and hence the gradient along eigenvector $\mathbf{u}_i$ is shrunk (or grown) by the factor λ_i^t. Architectural decisions, as well as appropriate regularization, can deal with vanishing gradients.